

Linux / RHEL

Interview & Exam Preparation — Q&A Guide

Theory + Practical questions across deployment, administration, security, networking, packaging, and monitoring topics

Prepared for	Md — DevOps/Linux Internship, Prodevans Technologies
---------------------	--

Scope	RHEL 9 administration, security, and infrastructure topics
--------------	--

Format	Theory + Practical (hands-on command) style questions
---------------	---

Table of Contents

1. Deployment (RHEL Installation) (8 questions)
2. User & Group Management (10 questions)
3. Permissions (Basic) (7 questions)
4. Advanced Permissions (ACL) (7 questions)
5. Special Permissions (SUID, SGID, Sticky Bit) (6 questions)
6. Package Management (7 questions)
7. Repository Server / Repository Client (7 questions)
8. Networking (7 questions)
9. Firewall (firewalld) (7 questions)
10. SELinux (7 questions)
11. Red Hat Satellite (6 questions)
12. Web Server (Apache httpd) (6 questions)
13. Migration (6 questions)
14. Monitoring through Prometheus (7 questions)
15. Linux Upgrade: RHEL 9 to RHEL 10 (7 questions)

1. Deployment (RHEL Installation)

Theory

Q1. What are the different methods available to deploy/install RHEL?

RHEL can be deployed via: (1) Interactive GUI/TUI installer using Anaconda from DVD/USB ISO, (2) Kickstart automated installation, (3) PXE network boot installation, (4) Cloning a pre-built image/template (common in virtualization), and (5) Cloud image deployment (AWS AMI, Azure image, etc.).

Theory

Q2. What is Kickstart and why is it used?

Kickstart is a file-based automated installation method (ks.cfg) that answers all installer prompts in advance — partitioning, network, root password, packages, post-install scripts — so RHEL can be installed unattended and identically across many machines. It's essential for large-scale, repeatable deployments.

Practical

Q3. How do you generate a Kickstart file from an already-installed system?

After installation, Anaconda saves the file used at /root/anaconda-ks.cfg. You can copy and edit this file, or use `system-config-kickstart` (GUI tool) to build one from scratch.

Practical

Q4. Give an example of a minimal Kickstart snippet for partitioning and package selection.

```
lang en_US.UTF-8
keyboard us
timezone Asia/Kolkata
rootpw --plaintext redhat
authselect select sssd
network --bootproto=dhcp --device=eth0
zerombr
clearpart --all --initlabel
autopart
%packages
@core
vim-enhanced
%end
```

Theory

Q5. What is PXE boot and how does it fit into deployment?

PXE (Preboot Execution Environment) allows a machine to boot over the network using DHCP + TFTP to fetch a boot image and kernel, then pull a Kickstart file via HTTP/NFS/FTP. It enables completely hands-off, network-based OS deployment for bare-metal servers.

Theory

Q6. What partitioning schemes have you used during deployment (LVM vs standard)?

LVM (Logical Volume Manager) is generally preferred in production because it allows online resizing, snapshots, and flexible storage pooling across multiple disks. Standard partitioning is simpler but inflexible once deployed — resizing requires more downtime and planning.

Practical

Q7. Practical: How would you validate a Kickstart file for syntax errors before using it?

Use `ksvalidator ks.cfg` (from the pykickstart package) to check for syntax issues before attaching it to an installation, avoiding failed unattended installs.`

Theory

Q8. What post-installation steps do you typically automate right after deployment?

Registering with subscription-manager, setting hostname/timezone, configuring network (static IP/DNS), creating initial users, hardening SSH, applying baseline security (SELinux enforcing, firewalld rules), and enrolling into configuration management (Ansible/Satellite).

2. User & Group Management

Theory

Q1. What files store user and group information in Linux?

/etc/passwd (user account info), /etc/shadow (encrypted passwords + aging), /etc/group (group info), and /etc/gshadow (secure group info).

Theory

Q2. Explain each field in a line of /etc/passwd.

username:password(x):UID:GID:GECOS(comment):home_directory:login_shell. The 'x' means the real password hash is stored in /etc/shadow, not here.

Practical

Q3. Command to create a new user with a specific UID, home directory, and shell?

```
`useradd -u 1050 -d /home/shafeeulla -m -s /bin/bash shafeeulla`
```

-u sets UID, -m creates home directory, -s sets the login shell.

Practical

Q4. How do you create a group and add an existing user to it as a secondary group?

```
`groupadd devops`  
`usermod -aG devops shafeeulla`
```

-a (append) is critical — without it, usermod replaces all existing secondary groups.

Theory

Q5. Difference between primary group and secondary/supplementary group?

Every user has exactly one primary group (recorded in /etc/passwd, used for newly created files) but can belong to multiple secondary groups (recorded in /etc/group) for additional shared access.

Practical

Q6. How do you lock and unlock a user account without deleting it?

```
`passwd -l username` locks (prepends ! to the password hash); `passwd -u username` unlocks it.  
Alternatively `usermod -L` / `usermod -U`.
```

Practical

Q7. How do you set password aging policy for a user (max days, warning days)?

```
`chage -M 90 -W 7 username` sets max password age to 90 days with a 7-day warning before expiry.  
`chage -l username` lists current aging settings.
```

Practical

Q8. How do you delete a user along with their home directory and mail spool?

```
`userdel -r username` removes the home directory and mail spool along with the account.
```

Theory

Q9. What is the difference between `su` and `su -`?

`su username` switches user but keeps the current shell's environment. `su - username` starts a full login shell, sourcing the target user's profile and resetting environment variables (PATH, HOME, etc.).

Q10. How would you audit which users have UID 0 (root privileges)?

``awk -F: '$3 == 0 {print $1}' /etc/passwd`` — normally only 'root' should return here; any other entry is a serious security concern.

3. Permissions (Basic)

Theory

Q1. What are the three basic permission types in Linux and what do they mean on a directory vs a file?

Read (r), Write (w), Execute (x). On a file: r = view contents, w = modify, x = run as a program. On a directory: r = list contents, w = create/delete files inside, x = cd into it / traverse it.

Theory

Q2. How is permission represented in symbolic vs numeric (octal) form?

Symbolic: `rwrx-xr--` for owner/group/other. Numeric: each `rwX` maps to $4+2+1$, so `rwrx-xr--` = 754. The three digits represent owner, group, and others respectively.

Practical

Q3. Command to give owner full permission, group read+execute, others no permission?

``chmod 750 filename`` or symbolically ``chmod u=rwx,g=rx,o= filename``.

Practical

Q4. How do you change ownership (user and group) of a file in one command?

``chown username:groupname filename`` — changes both user and group owner in a single command. Use ``-R`` to apply recursively to a directory tree.

Theory

Q5. What does umask do and how is the default file permission calculated from it?

umask subtracts permissions from the maximum default (666 for files, 777 for directories). E.g., with umask 022, new files get $666-022=644$ and new directories get $777-022=755$.

Practical

Q6. How do you find all files in /var/log with permission 777 (world-writable)?

``find /var/log -type f -perm 777`` — useful for security audits to spot dangerously open files.

Theory

Q7. What's the difference between chmod applied recursively on files vs directories, and why is `chmod -R` sometimes risky?

``chmod -R`` applies the same mode to files AND directories, but files usually shouldn't get the execute bit while directories need it to be traversable. Using ``find ... -type d -exec chmod 755`` and ``find ... -type f -exec chmod 644`` separately avoids breaking directory traversal or making scripts unintentionally non-executable.

4. Advanced Permissions (ACL)

Theory

Q1. Why do we need ACLs (Access Control Lists) when we already have standard Unix permissions?

Standard permissions only support one owner, one group, and 'others' — a single rule set. ACLs allow you to grant different permissions to multiple specific users or groups on the same file/directory, which standard permissions cannot express.

Practical

Q2. How do you check if a filesystem supports ACLs, and how do you view ACLs on a file?

The filesystem must be mounted with the ``acl`` option (default on most modern ext4/xfs mounts). View ACLs with ``getfacl filename``.

Practical

Q3. Command to give a specific user 'rw' access to a file via ACL, without changing the owner/group?

``setfacl -m u:john:rw- /data/report.txt`` grants user john read+write access specifically, on top of the standard permission bits.

Practical

Q4. How do you set a default ACL on a directory so all new files inside inherit it?

``setfacl -d -m g:devops:rwx /shared/project`` sets a default ACL — any new file/subdirectory created inside `/shared/project` afterward automatically inherits devops group rwx access.

Practical

Q5. How do you remove all ACL entries from a file?

``setfacl -b filename`` removes all extended ACL entries, reverting to standard owner/group/other permissions only.

Theory

Q6. When you run ``ls -l`` on a file that has an ACL applied, how can you tell?

A '+' appears after the permission string, e.g. ``-rw-rwxr--+``, signaling that extended ACL entries exist beyond the basic permission bits shown.

Practical

Q7. How do you copy ACLs from one file to another?

``getfacl file1 | setfacl --set-file=- file2`` pipes the ACL entries of file1 and applies them identically to file2.

5. Special Permissions (SUID, SGID, Sticky Bit)

Theory

Q1. What is SUID and give a real example of where it's used by default?

SUID (Set User ID) makes an executable run with the file owner's privileges rather than the invoking user's. Example: `/usr/bin/passwd` is owned by root with SUID set, so a normal user can update `/etc/shadow` (which only root can write) when changing their own password.

Theory

Q2. What is SGID and how does its effect differ on a file vs a directory?

On a file, SGID makes it execute with the group owner's privileges. On a directory, SGID makes all new files/subdirectories created inside automatically inherit the directory's group — heavily used for shared team/project directories.

Theory

Q3. What is the Sticky Bit and where is it classically applied?

Sticky bit on a directory restricts file deletion so only the file's owner (or root) can delete/rename it, even if others have write access to the directory. Classic example: `/tmp` has sticky bit set so users can't delete each other's temp files.

Practical

Q4. Give the chmod numeric and symbolic commands to set SUID, SGID, and sticky bit.

SUID: ``chmod u+s file`` or ``chmod 4755 file``

SGID: ``chmod g+s dir`` or ``chmod 2755 dir``

Sticky: ``chmod +t dir`` or ``chmod 1777 dir``

The leading digit encodes special perms: 4=SUID, 2=SGID, 1=sticky (can be combined, e.g. 6755).

Practical

Q5. How do you identify all SUID binaries on a system (common security audit task)?

``find / -perm -4000 -type f 2>/dev/null`` lists all SUID files; compare against a known baseline to detect suspicious/injected SUID binaries.

Theory

Q6. How is the special permission shown in ``ls -l`` output for SUID/SGID/sticky bit?

SUID shows as 's' in the owner execute position (rwsr-xr-x); SGID shows as 's' in the group execute position (rwxr-sr-x); Sticky bit shows as 't' in the other execute position (rwxrwxrwt). If the underlying execute bit wasn't set, it appears as uppercase S or T instead.

6. Package Management

Theory

Q1. What's the difference between rpm and dnf/yum?

rpm is the low-level package tool that installs/queries/verifies individual .rpm files but does NOT resolve dependencies automatically. dnf (yum's successor since RHEL 8) is the high-level tool that resolves dependencies automatically by querying configured repositories.

Practical

Q2. Command to install a package, and to install from a local .rpm file with dependency resolution?

From repo: `dnf install httpd``

Local rpm with deps resolved from repo: `dnf install ./package.rpm``

(Using `rpm -ivh package.rpm`` directly will fail if dependencies aren't already present.)

Practical

Q3. How do you check which package provides a specific file/command?

`dnf provides /usr/bin/vim`` or `rpm -qf /usr/bin/vim`` — useful when troubleshooting missing binaries.

Practical

Q4. How do you list all files installed by a package, and verify none have been tampered with?

List files: `rpm -ql packagename``

Verify integrity: `rpm -V packagename`` — compares checksums, permissions, size, etc. against the RPM database; any deviation (like a modified config or binary) is flagged.

Theory

Q5. What are RPM package groups and modules (AppStream) in RHEL 8+?

Modules (via AppStream) let you install different versions/streams of the same software (e.g., PHP 7.4 vs 8.1) side by side in the metadata sense, chosen with `dnf module install php:8.1``. This solved the 'only one version per RPM release' problem of earlier RHEL.

Practical

Q6. How do you downgrade a package or roll back after a bad dnf update?

`dnf history`` shows past transactions; `dnf history undo`` reverses a specific transaction. `dnf downgrade packagename`` installs an older cached/repo version directly.

Practical

Q7. How do you check for and apply only security-related updates?

`dnf updateinfo list security`` lists security advisories; `dnf update --security`` applies only packages tied to security errata (requires yum-plugin-security equivalent, built into dnf).

7. Repository Server / Repository Client

Theory

Q1. What is the purpose of setting up a local/internal repository server?

It avoids every client needing internet access to Red Hat's CDN, saves bandwidth, ensures consistent package versions across the fleet, and is required in air-gapped/offline environments common in enterprise and government networks.

Practical

Q2. How do you set up a basic HTTP-based repo server for RPM packages?

1. ``dnf install httpd createrepo_c``
2. Copy RPMs into e.g. `/var/www/html/repo/`
3. Run ``createrepo_c /var/www/html/repo/`` to generate the repodata/ metadata
4. ``systemctl enable --now httpd`` and open firewall for port 80

Clients then point a .repo file's baseurl at `http://server/repo`.

Theory

Q3. What does createrepo_c actually do?

It scans a directory of .rpm files and generates the repodata/ metadata (primary.xml, filelists.xml, repomd.xml, etc.) that dnf/yum need to resolve dependencies and know what packages exist in that repo.

Practical

Q4. Show a sample client-side .repo file pointing to your internal repo.

```
[internal-baseos]
name=Internal BaseOS Repo
baseurl=http://repo.local/rhel9/baseos
enabled=1
gpgcheck=0
```

Placed in `/etc/yum.repos.d/internal.repo` on the client.

Practical

Q5. How do you update repo metadata after adding new RPMs to an existing repo?

``createrepo_c --update /var/www/html/repo/`` regenerates metadata incrementally instead of scanning everything from scratch.

Practical

Q6. How do you configure gpgcheck properly for a trusted internal repo (best practice, not gpgcheck=0)?

Import the signing key on clients (``rpm --import /path/to/RPM-GPG-KEY``), then set ``gpgcheck=1`` and ``gpgkey=http://repo.local/RPM-GPG-KEY`` in the .repo file so packages are verified before install.

Practical

Q7. How would you troubleshoot a client that can't reach the repo (dnf 'Failed to download metadata')?

Check network reachability (``curl -I http://repo.local/repo/repodata/repomd.xml``), firewall rules on both ends, httpd service status on the server, correct baseurl/syntax in the .repo file, and SELinux context on the repo directory (``httpd_sys_content_t``).

8. Networking

Theory

Q1. What tool is the modern standard for managing network configuration in RHEL 8/9?

NetworkManager, managed via `nmcli` (CLI) or `nmtui` (text UI). Legacy `ifcfg`-style scripts under `/etc/sysconfig/network-scripts/` are still supported but `nmcli` is the recommended interface.

Practical

Q2. How do you configure a static IP on an interface using nmcli?

```
nmcli con mod eth0 ipv4.addresses 192.168.1.50/24
nmcli con mod eth0 ipv4.gateway 192.168.1.1
nmcli con mod eth0 ipv4.dns 8.8.8.8
nmcli con mod eth0 ipv4.method manual
nmcli con up eth0
```

Practical

Q3. How do you create a new connection profile bound to a specific interface?

```
`nmcli con add type ethernet ifname ens192 con-name lan-static ipv4.addresses 10.0.0.5/24 ipv4.gateway 10.0.0.1 ipv4.method manual`
```

Practical

Q4. How do you check current IP, routing table, and DNS resolution respectively?

IP: ``ip addr show``

Routes: ``ip route show``

DNS: ``cat /etc/resolv.conf`` or ``resolvectl status`` (systemd-resolved) or ``nmcli dev show | grep DNS``.

Theory

Q5. What's the difference between a bridge and a bond interface?

A bond aggregates multiple NICs for redundancy/throughput to behave as one logical interface (active-backup, LACP, etc.). A bridge connects multiple network segments/interfaces at Layer 2, commonly used to let VMs share a host's physical NIC.

Practical

Q6. How do you troubleshoot 'no route to host' between two RHEL servers?

Check: (1) ``ping`` and ``traceroute`` to isolate where connectivity breaks, (2) ``ip route`` on both ends, (3) `firewalld` rules (``firewall-cmd --list-all``), (4) that the interface is actually up (``nmcli dev status``), and (5) any intermediate network ACLs/security groups if in cloud/virtualized env.

Practical

Q7. How do you make a hostname change persistent?

``hostnamectl set-hostname newname.example.com`` — updates `/etc/hostname` persistently (unlike just running ``hostname newname`` which only affects the current session).

9. Firewall (firewalld)

Theory

Q1. What is firewalld and how does it differ conceptually from raw iptables?

firewalld is a dynamic firewall management daemon with a D-Bus interface that manages rules through 'zones' (trust levels) and 'services', without needing to reload the entire ruleset for each change (unlike traditional iptables where changes required a full ruleset reload).

Theory

Q2. What is a 'zone' in firewalld and name a few common ones?

A zone defines the trust level for a set of network connections/interfaces. Common zones: `public` (default, distrusted), `trusted` (allow all), `internal` (for internal networks), `dmz` (limited public-facing access), `drop` (silently drop everything incoming).

Practical

Q3. How do you permanently open port 8080/tcp and reload firewalld?

```
`firewall-cmd --permanent --add-port=8080/tcp`
```

```
`firewall-cmd --reload`
```

Without `--permanent`, the rule only applies to the running config and is lost on reboot/reload.

Practical

Q4. How do you allow a specific service (like http) instead of a raw port?

```
`firewall-cmd --permanent --add-service=http`
```

```
`firewall-cmd --reload`
```

Using named services is cleaner and self-documenting versus remembering raw port numbers.

Practical

Q5. How do you check which zone an interface belongs to, and change it?

Check: ``firewall-cmd --get-zone-of-interface=eth0``

Change: ``firewall-cmd --zone=internal --change-interface=eth0 --permanent``

Practical

Q6. How do you allow access to a service only from a specific source IP/subnet (rich rule)?

```
`firewall-cmd --permanent --zone=public --add-rich-rule='rule family="ipv4" source address="192.168.1.0/24" service name="ssh" accept`
```

```
`firewall-cmd --reload`
```

Practical

Q7. How do you list all currently active rules for the default zone?

``firewall-cmd --list-all`` shows services, ports, rich rules, masquerading status, etc. for the active default zone.

10. SELinux

Theory

Q1. What are the three SELinux modes and what does each do?

Enforcing: policy violations are blocked and logged. Permissive: violations are only logged, not blocked (useful for testing/troubleshooting). Disabled: SELinux is completely off (not recommended in production).

Practical

Q2. How do you check the current SELinux mode and change it temporarily vs permanently?

Check: ``getenforce`` or ``sestatus``

Temporary (until reboot): ``setenforce 0`` (permissive) / ``setenforce 1`` (enforcing)

Permanent: edit ``SELINUX=enforcing|permissive|disabled`` in `/etc/selinux/config`, requires reboot.

Practical

Q3. A web server can't read files even though standard Unix permissions look correct — likely cause and diagnosis?

Likely an SELinux context mismatch (e.g., file has `default_t` instead of `httpd_sys_content_t`). Diagnose with ``ls -Z filename`` to view its context, and check `/var/log/audit/audit.log`` or ``ausearch -m avc -ts recent`` for denial messages.

Practical

Q4. How do you fix an SELinux context issue for files served by Apache in a non-standard directory?

``semanage fcontext -a -t httpd_sys_content_t '/data/web(/.*)?``

``restorecon -Rv /data/web``

The first command adds a persistent rule to the SELinux file context database; `restorecon` applies it.

Theory

Q5. What is the difference between ``chcon`` and ``semanage fcontext`` + ``restorecon``?

``chcon`` changes the context immediately but is NOT persistent — a relabel or ``restorecon`` run will revert it. ``semanage fcontext`` adds a rule to the persistent policy database, and ``restorecon`` applies it; this survives relabeling and reboots, making it the correct approach for permanent fixes.

Practical

Q6. What is an SELinux boolean and how do you toggle one?

Booleans are on/off switches that adjust policy behavior without writing custom policy modules, e.g.

``httpd_can_network_connect`` allows Apache to make outbound network connections (needed for reverse proxies). Toggle with ``setsebool -P httpd_can_network_connect on`` (-P makes it persistent).

Practical

Q7. How do you use `audit2allow` to build a custom policy module from denials?

``ausearch -m avc -ts recent | audit2allow -M mypolicy``

``semodule -i mypolicy.pp``

This generates and loads a custom policy module permitting exactly the actions that were denied — used cautiously, only after confirming the denial is a legitimate need and not a real security issue.

11. Red Hat Satellite

Theory

Q1. What problem does Red Hat Satellite solve at enterprise scale?

It provides centralized subscription management, patch/content lifecycle management, provisioning, and configuration management for large fleets of RHEL systems — instead of every server registering individually with Red Hat's CDN and managing updates independently.

Theory

Q2. What are the core building blocks of Satellite's content management model?

Organizations > Products > Repositories > Content Views > Lifecycle Environments. Content Views let you snapshot a curated, versioned set of repos; Lifecycle Environments (e.g. Dev -> Test -> Prod) let you promote a tested content view through stages in a controlled way.

Theory

Q3. What is a Capsule Server and why is it used?

A Capsule Server is a proxy/content mirror deployed at remote sites/datacenters so that client systems don't need direct connectivity to the central Satellite server for content sync, DNS, DHCP, TFTP, or Puppet — improving performance and reducing WAN dependency.

Practical

Q4. How do you register a RHEL client to Satellite?

Typically via the bootstrap script generated per Activation Key from the Satellite web UI, e.g.

```
`subscription-manager register --org="MyOrg" --activationkey="rhel9-key"`
```

or running the Satellite-provided bootstrap.py script which also installs katello-agent/host tools.

Theory

Q5. What is an Activation Key used for in Satellite?

It bundles a Content View, Lifecycle Environment, subscriptions, and host group settings into one reusable token, so new hosts register with the correct content and entitlements automatically in one step.

Practical

Q6. How would you promote content (patches) from Dev to Prod in a controlled way using Satellite?

Publish a new Content View version, then use 'Promote' in the Satellite UI (or hammer CLI: ``hammer content-view version promote --content-view "MyCV" --to-lifecycle-environment "Prod"``) after it has been validated in earlier lifecycle environments like Dev/QA.

12. Web Server (Apache httpd)

Practical

Q1. How do you install and enable Apache (httpd) on RHEL, then verify it's running?

```
`dnf install httpd`
```

```
`systemctl enable --now httpd`
```

```
`systemctl status httpd`
```

Also open the firewall: ``firewall-cmd --permanent --add-service=http && firewall-cmd --reload``.

Theory

Q2. What is the main configuration file and default document root for httpd on RHEL?

Main config: `/etc/httpd/conf/httpd.conf` (plus drop-in configs under `/etc/httpd/conf.d/`). Default

DocumentRoot: `/var/www/html`.

Practical

Q3. How do you configure a virtual host to serve a second website on the same server?

Create a new file e.g. `/etc/httpd/conf.d/mysite.conf`:

```
<VirtualHost *:80>
  ServerName mysite.example.com
  DocumentRoot /var/www/mysite
  ErrorLog /var/log/httpd/mysite-error.log
  CustomLog /var/log/httpd/mysite-access.log combined
</VirtualHost>
```

Then ``systemctl restart httpd`` after fixing SELinux context/firewall as needed.

Practical

Q4. A freshly configured vhost shows '403 Forbidden' even though the file exists with correct Unix permissions — likely causes?

Two usual suspects: (1) SELinux context on the new DocumentRoot isn't `httpd_sys_content_t` (fix with `semanage fcontext + restorecon`), or (2) missing `'index.html/DirectoryIndex` and no block granting ``Require all granted`` for that path.

Practical

Q5. How do you enable SSL/TLS on Apache and where are the typical cert paths?

``dnf install mod_ssl``, then edit `/etc/httpd/conf.d/ssl.conf` pointing `SSLCertificateFile` and `SSLCertificateKeyFile` to your cert/key (commonly under `/etc/pki/tls/certs` and `/etc/pki/tls/private`). Restart `httpd` and open port 443 in `firewalld`.

Practical

Q6. How do you check Apache configuration syntax before restarting the service in production?

``httpd -t`` (or ``apachectl configtest``) validates syntax without applying changes — critical to avoid taking down a live web server with a typo.

13. Migration

Theory

Q1. What are P2V and V2V migrations?

P2V (Physical-to-Virtual): converting a workload running on physical hardware into a VM image. V2V (Virtual-to-Virtual): converting a VM from one hypervisor format to another (e.g., VMware VMDK to KVM qcow2, common when migrating off VMware ESXi to Red Hat virtualization/OpenShift Virtualization).

Practical

Q2. What tool does Red Hat provide for V2V conversions, and what's a basic usage example?

``virt-v2v`` converts VM disk images/configs between hypervisors. Example:

```
`virt-v2v -i libvirtxml vm.xml -o local -os /var/lib/libvirt/images`
```

It handles driver injection (e.g., replacing VMware tools with virtio drivers) so the VM boots correctly on the new hypervisor.

Theory

Q3. What key things do you validate before migrating a production RHEL server to new infrastructure?

Hardware/driver compatibility (especially storage/network drivers), matching kernel version support, subscription/entitlement re-registration, network config (IP/hostname/DNS) portability, backup of current state/snapshot as rollback point, and a tested rollback plan.

Practical

Q4. How would you migrate application data with minimal downtime (e.g., moving a database's data directory to new storage)?

Use `rsync` for an initial bulk copy while the service is still running (``rsync -avz --progress /data/ /newstorage/data/``), then do a short final ``rsync -avz --delete`` pass during a brief maintenance window to sync only the incremental changes before cutting over — minimizing actual downtime.

Theory

Q5. What's the difference between a 'lift and shift' migration and a re-platforming migration?

Lift-and-shift moves the workload as-is (same OS/config) onto new infrastructure with minimal changes — fast but may carry over technical debt. Re-platforming intentionally modernizes during migration (e.g., moving from a manually configured VM to a containerized/Kubernetes-managed deployment).

Practical

Q6. How do you handle subscription-manager re-registration after migrating/cloning a RHEL system?

Clean the old system's UUID first: ``subscription-manager clean`` (or unregister on the source before cloning), then ``subscription-manager register --org=... --activationkey=...`` on the new/cloned system to avoid UUID conflicts and duplicate/orphaned entitlement entries in the subscription portal.

14. Monitoring through Prometheus

Theory

Q1. At a high level, how does Prometheus monitoring work?

Prometheus is a pull-based time-series monitoring system: it periodically scrapes metrics endpoints (usually an HTTP /metrics path) exposed by 'exporters' running on target hosts, stores the data as time series, and lets you query it with PromQL. Alertmanager handles alert routing/notification separately from the core Prometheus server.

Practical

Q2. What is 'node_exporter' and how do you set it up on a RHEL host?

node_exporter exposes host-level OS metrics (CPU, memory, disk, network) on port 9100 for Prometheus to scrape. Setup: download the binary, run it as a systemd service, then add the host as a scrape target in prometheus.yml under `scrape\_configs`.

Practical

Q3. Give a minimal prometheus.yml scrape_config snippet to monitor two RHEL servers via node_exporter.

```
yaml
scrape_configs:
- job_name: 'rhel_nodes'
  static_configs:
  - targets: ['server1:9100', 'server2:9100']
```

After editing, reload with `curl -X POST http://localhost:9090/-/reload` or restart the service.

Practical

Q4. What is PromQL and give an example query to check CPU usage percentage?

PromQL is Prometheus's query language for time-series data. Example to get average CPU usage % over the last 5 minutes:

```
`100 - (avg by(instance)(rate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)`
```

Practical

Q5. How do you validate that Prometheus is actually scraping a target successfully?

In the Prometheus web UI, go to Status > Targets — it shows each target's state (UP/DOWN), last scrape time, and error message if scraping failed (e.g., connection refused, timeout).

Theory

Q6. What's the difference between Prometheus and Grafana in a typical monitoring stack?

Prometheus is the metrics collection, storage, and alerting engine (the data layer). Grafana is the visualization layer — it queries Prometheus (and other data sources) via PromQL to build dashboards; Grafana itself does not store metrics.

Practical

Q7. How would you set up a basic alert rule (e.g., high memory usage) in Prometheus?

Define an alerting rule file:

```
yaml
groups:
- name: memory_alerts
```

```
rules:
- alert: HighMemoryUsage
  expr: node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes < 0.10
  for: 5m
  labels:
  severity: critical
  annotations:
  summary: "Available memory below 10% on {{ $labels.instance }}"
```

Reference this rule file in prometheus.yml under `rule\_files`, and Alertmanager handles notification routing.

15. Linux Upgrade: RHEL 9 to RHEL 10

Theory

Q1. What official Red Hat tool is used for in-place major version upgrades (e.g. RHEL 9 -> 10)?

Leapp (``leapp-upgrade`` utility). It analyzes the current system, checks for known upgrade blockers (incompatible packages, custom repos, kernel modules, deprecated config), and performs the actual in-place OS upgrade rather than requiring a fresh install.

Practical

Q2. What is the general workflow to upgrade a RHEL 9 system to RHEL 10 using Leapp?

1. ``dnf install leapp-upgrade``
2. ``leapp preupgrade`` — runs a dry-run risk/compatibility assessment, generating a report
3. Review `/var/log/leapp/leapp-report.txt` and resolve any 'inhibitor' issues it flags
4. ``leapp upgrade`` — prepares the upgrade (downloads target packages, builds an upgrade initramfs)
5. Reboot — the system boots into a special upgrade environment, performs the actual OS swap, then reboots again into RHEL 10.

Practical

Q3. What are common 'inhibitors' that block a Leapp upgrade, and how do you handle them?

Typical inhibitors: unsupported/EOL third-party repos still enabled, deprecated kernel drivers or modules, custom or unsupported partition layouts, and packages with no upgrade path. You resolve them by removing/disabling the offending repo or package, or explicitly acknowledging via Leapp answer files (``leapp answer``) only when you understand and accept the risk.

Theory

Q4. Why should you always take a full backup/snapshot before running an in-place major upgrade?

In-place upgrades modify the running OS irreversibly — kernel, core libraries, and system config all change. If something breaks mid-way or a critical incompatibility surfaces post-upgrade, a backup/VM snapshot is the only reliable rollback path since Leapp itself doesn't offer a clean 'undo' after the swap completes.

Theory

Q5. What should you check regarding third-party software (e.g., monitoring agents, custom RPMs) before upgrading?

Confirm each vendor explicitly supports the target RHEL major version — kernel ABI, driver, and library changes across major versions frequently break out-of-tree kernel modules (like some storage/network drivers) and vendor agents compiled against older glibc/openssl versions.

Practical

Q6. After the upgrade completes, what should you verify first?

``cat /etc/redhat-release`` to confirm the new version, ``systemctl --failed`` to check for services that didn't come back up cleanly, SELinux status (``sestatus``) since policy versions change across majors, and re-run ``subscription-manager`` status/repo checks since entitlements may need refreshing for the new major version's repos.

Q7. Is upgrading RHEL 9 to 10 in-place always the recommended path, or when would you instead do a fresh install + migration?

For critical production systems, many organizations prefer a fresh RHEL 10 install plus application/data migration (cleaner state, no legacy cruft) over an in-place Leapp upgrade, especially when heavy customization or unsupported components make Leapp's blocker list too large to safely resolve. In-place upgrade is favored when uptime/config continuity matters more and the system is relatively vanilla.